Divisão de Trabalho — Tower Defense (Unity)

Projeto TD · Documento operacional · Baseado na análise do repositório (~88 scripts C#) e no *OrdemDesenvolvimento_TowerDefense* · Gerado para alinhar responsabilidades em duas frentes: **inimigo + estatísticas** vs **aliado / defesa / economia**.

1. Objetivo desta divisão

Separar o desenvolvimento em **duas fases paralelas** (não necessariamente sequenciais no tempo), cada uma com um responsável principal, para reduzir conflitos de merge e clarificar escopo: tudo que é **ameaça ao jogador** (ondas, criaturas, chefes, catálogos) fica com um dev; tudo que é **ferramenta de defesa e economia do jogador** (torres, heróis, coleta, eventos de apoio) fica com outro. Itens **transversais** (save, perfil, cenas, núcleo da partida) exigem acordo rápido em contratos (dados e eventos).

2. Panorama do código (resumo)

Núcleo compartilhado: `WaveManager`, `GameManager`, `PerfilJogadorAtivo`, `SaveSlotManager` / `DadosSlotSave`, `OuroManager`, `LootManager`, rotas e spawn. **Já existente:** progressão de raça (EXP), relíquias (base), inventário, vitória na wave 8, filtro de raça inimiga. **Ainda inexistente ou mínimo:** sistema de torres, heróis em campo, eventos de contratação/comércio, bestiário/bossiário/talentos e estatísticas globais conforme design doc.

3. Responsável A — Parte inimiga + estatísticas (Jonas)

> **Escopo principal**
>
> - **Ondas e ameaça:** configuração e balanceamento de waves (`WaveManager`, `WaveRacaConfig`, prefabs, bosses nas ondas 3/6/8), novos inimigos e variantes por raça.
> - **Unidades inimigas:** controladores em `Racas/`, projéteis, bosses (`Bosses/`), `UnidadeController` / `MovimentoUnidade`, morte e listeners (`IMorteListener`, `ExpAoMorrerInimigo`, `DropLootAoMorrer`).
> - **Bestiário:** registro de espécies enfrentadas, desbloqueio progressivo, UI de consulta.
> - **Bossiário:** registro e fichas de chefes (níveis/variações), separado ou integrado ao bestiário.
> - **Estatísticas:** agregação de dados de partida (kills, dano recebido/causedo, tempo, etc.) e telas de resumo; integração com save se necessário.
> - **Árvore de talentos (eixo combate/meta):** conforme design — talentos ligados a progresso de conta ou conhecimento de inimigos; definição de dados e UI se for desta frente.
>
> **Ordem sugerida (esta frente):** **1)** fechar catálogo mínimo de inimigos + bosses por documento · **2)** dados de bestiário/bossiário persistidos no perfil · **3)** estatísticas de fim de partida e/ou histórico · **4)** árvore de talentos alinhada aos dados salvos.

4. Responsável B — Parte aliada / defesa / economia (outro dev)

> **Escopo principal**

> - **Torres:** colocação, grid/slots, custos, ataque a inimigos, upgrades, venda (regras do design doc).
> - **Heróis:** evolução de `HeroisManager`, spawn em campo, comportamento, limite de 3 por partida.
> - **Eventos:** JOB HUNTER, MERCHANT e fluxos associados (UI + ouro).
> - **Coleta / minérios:** `MinerioDeOuro`, `RecursoController`, `OuroManager`, integração com mapa e HUD.
> - **Unidades aliadas jogáveis** (ex.: humanos no mapa), painéis existentes (`PainelUnidadesController`, botões), onde couber o design.
> - **Castelo / estrutura do jogador** como defesa (HP, upgrades futuros), em coordenação com o núcleo.
>
> **Ordem sugerida (esta frente):** **1)** torre "mínima jogável" (colocar + atirar) · **2)** economia mineiro + ouro · **3)** heróis em campo · **4)** eventos e refinamento de coleta.

5. Zona compartilhada (combinar antes de codar)

- **Save:** novos campos em `DadosSlotSave` — quem adiciona qual lista (prefixo ou módulos claros).
- **Eventos globais:** ex. `SaveSlotManager.OnDadosPerfilPersistidos`, fim de partida, vitória/derrota.
- **Cenas e fluxo:** lobby → mapa; evitar dois prefabs alterando o mesmo Canvas sem combinar.
- **Interfaces / IDs:** IDs de tipo de inimigo, torre e item — uma convenção (strings ou ScriptableObjects).

6. Matriz rápida (quem cuida do quê)

Área	Principal A (inimigo + stats)	Principal B (aliado)
Waves / spawn inimigo	●	○ integração mapa
Prefabs inimigos / boss	●	○ alvo das torres
Torres	○ alvo	●
Heróis / eventos	○	●
Minérios / ouro jogador	○ drops inimigo	● coleta / HUD
Bestiário / bossiário / talentos	●	○
Estatísticas (telas)	●	○ torre/herói se compartilhar tela
Save / perfil	ambos — contrato	

○ = apoio ou consumidor · ● = dono principal da feature.

Nota: Esta divisão não substitui o *OrdemDesenvolvimento_TowerDefense.pdf*; apenas distribui o trabalho entre duas pessoas. Ajustem nomes e ordem interna conforme sprint.